

Documento Técnico y Ejecutivo — StoreFlow

StoreFlow es una plataforma web moderna, modular y de alto rendimiento diseñada para la **gestión comercial integral, administración de Punto de Venta (POS), control de inventarios y auditoría operativa en tiempo real** para negocios minoristas y de comercio al detal.

1. Planteamiento del Problema

Los negocios comerciales (tiendas de abarrotes, minimarkets, boutiques, ferreterías y comercios minoristas) enfrentan desafíos cotidianos que afectan directamente su rentabilidad y control operativo:

1. **Descontrol en la Caja Registradora:** Ventas procesadas sin apertura previa de turno, descuadres en los arqueos de efectivo al final de la jornada y falta de trazabilidad en los ingresos y egresos.
 2. **Pérdida y Descuadres de Inventario:** Dificultad para rastrear entradas, salidas, mermas o ajustes manuales, sumado a la lentitud en la codificación de productos (SKUs largos o complejos).
 3. **Vulnerabilidad ante Caídas de Red / Servidores:** Pérdida de ventas cuando el internet falla o la base de datos en la nube está en mantenimiento.
 4. **Procesos Lentos en Caja (POS):** Exigir datos del cliente de forma obligatoria en ventas de mostrador genera filas y demoras innecesarias.
 5. **Duplicidad de Clientes:** Registro repetido de clientes por no validar adecuadamente el número de Cédula o NIT con normalización de caracteres.
 6. **Falta de Auditoría y Trazabilidad:** Imposibilidad de saber qué usuario realizó una modificación en el sistema, anuló una venta o alteró el stock de un producto.
-

2. La Solución StoreFlow

StoreFlow resuelve de raíz estas problemáticas mediante una plataforma intuitiva, ágil y robusta:

- **Arquitectura Resiliente Offline-First con Health Check:** Capacidad de operar y cobrar sin interrupción ante cortes de internet o caídas directas de Supabase mediante `syncService.ts`.
- **Control Estricto de Caja (Opción A):** Bloqueo total de cobros si la caja registradora está cerrada, ofreciendo una ventana flotante de apertura rápida en 1 solo clic.
- **Punto de Venta Ágil:** Permite procesar ventas de contado a "*Público general*" sin exigir cliente, pero exige cliente de forma estricta únicamente para ventas a crédito.
- **Generación Inteligente de SKU:** Algoritmo que auto-crea SKUs cortos y legibles con abreviaciones de 3 letras (ej. `COC-COL-600ML`).

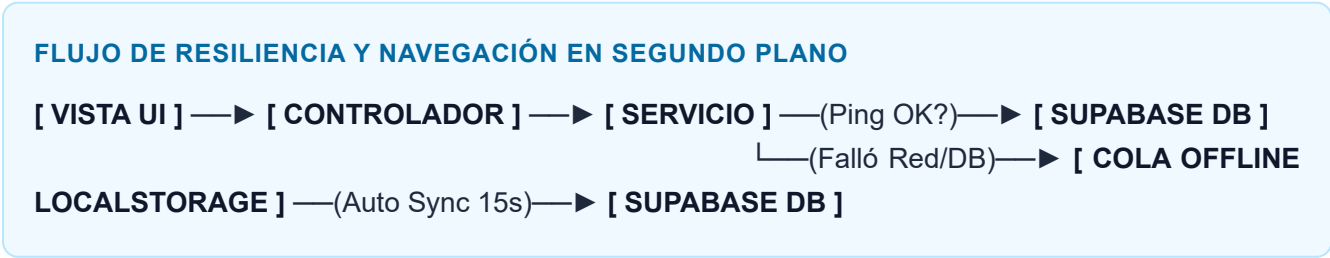
- **Validación de Cédula/NIT Normalizada:** Limpieza de puntos, guiones y espacios para evitar duplicados en la base de datos de clientes.
- **Bitácora de Auditoría 100% Automatizada:** Registro silencioso en Supabase (`activity_logs`) de cada inicio de sesión, venta, movimiento de caja, ajuste de stock y edición de usuarios.

3. Arquitectura del Sistema (Patrón MVC + Capa de Servicios & Motor Offline)

StoreFlow implementa el patrón de diseño **MVC (Modelo - Vista - Controlador)** evolucionado con una **Capa de Servicios (Service Layer Pattern)** y un **Motor de Sincronización Resiliente (Offline-First Engine)**. Esta arquitectura garantiza la separación física y lógica de responsabilidades, la alta disponibilidad y la tolerancia a fallos.

3.1 Mapeo de la Arquitectura MVC en StoreFlow

Componente MVC	Ubicación en el Código	Responsabilidad y Detalle Técnico
1. VISTA (View Layer)	<code>src/views/pages/*.tsx</code> <code>src/views/components/*.tsx</code>	Renderizado de la interfaz gráfica (UI/UX) con React 18, Tailwind CSS, Framer Motion y Lucide Icons. Captura de eventos del usuario sin contener lógica de almacenamiento.
2. CONTROLADOR (Controller Layer)	<code>src/controllers/*.ts</code> <code>StoreController.tsx</code>	Orquestación del estado de la aplicación, ejecución de reglas de negocio, permisos de usuarios (RBAC) y temporizador de fondo (<i>heartbeat</i>) para auto-sincronización.
3. SERVICIOS & RESILIENCIA (Service & Sync Layer)	<code>src/services/*.ts</code> <code>syncService.ts</code>	Capa de I/O y resiliencia offline. Incluye sondeo de salud (<i>Health Check Ping</i> a Supabase) y cola de pendientes (<code>pending_sync_queue</code>) en <code>LocalStorage</code> .
4. MODELO DE DATOS (Model Layer)	<code>src/models/types.ts</code> <code>supabase/schema.sql</code>	Contrato de datos en TypeScript, esquemas de tablas relacionales de PostgreSQL y cliente singleton oficial de Supabase.



4. Estructura Detallada del Proyecto

```

StoreFlow/
├── src/
│   ├── controllers/                # CAPA 2: Controladores de Estado y Reglas
│   │   ├── AuthController.ts      # Sesiones, login, usuarios y auditoría de acceso
│   │   ├── CashController.ts      # Turnos de caja, montos base y movimientos
│   │   ├── CustomerController.ts  # Clientes, cartera a crédito y abonos
│   │   ├── ProductController.ts   # Productos, categorías y control de stock
│   │   ├── PurchaseController.ts  # Proveedores, compras e ingreso a bodega
│   │   ├── SalesController.ts     # Checkout POS, cálculo de totales y anulaciones
│   │   ├── SettingsController.ts  # Configuración de la empresa y temas UI
│   │   ├── StoreController.tsx    # Orquestador central y temporizador de Auto-Sync
│   │   ├── permissions.ts        # Matriz RBAC de permisos (Supervisor vs Cajero)
│   │   └── types.ts              # Contrato del StoreContextValue
│   ├── services/                  # CAPA 3: Servicios puros de I/O y Sincronización
│   │   ├── authService.ts        # I/O para la tabla app_users
│   │   ├── cashService.ts        # I/O para cash_sessions y cash_movements
│   │   ├── customerService.ts    # I/O para la tabla customers
│   │   ├── productService.ts    # I/O para products, categories, inventory_adjustments
│   │   ├── purchaseService.ts    # I/O para suppliers, purchases, purchase_items
│   │   ├── salesService.ts       # I/O para sales y sale_items
│   │   ├── settingsService.ts    # I/O para company_settings y activity_logs
│   │   ├── syncService.ts        # Motor de resiliencia, cola offline y Health Check
│   │   └── index.ts
│   ├── models/                   # CAPA 4: Definición del Modelo de Datos
│   │   ├── types.ts              # Interfaces TypeScript (Product, Sale, Customer, etc.)
│   │   ├── seed.ts               # Semilla de datos iniciales
│   │   └── supabase.ts           # Cliente singleton de Supabase
│   ├── views/                    # CAPA 1: Vistas de Interfaz de Usuario
│   │   ├── pages/                # POSPage, CashPage, ProductsPage, SettingsPage, etc.
│   │   ├── components/layout/    # AppLayout, Sidebar, Topbar
│   │   └── components/ui/        # Button, Input, Select, Card, Dialog, DataTable
│   └── lib/
│       └── utils.ts              # Algoritmo de SKU, ID secuenciales, formateadores
├── supabase/
│   └── schema.sql                # Script DDL de tablas relacionales PostgreSQL
├── package.json
└── vite.config.ts
  
```

5. Modelo de Datos Relacional en Supabase (PostgreSQL)

StoreFlow utiliza las siguientes tablas relacionales en Supabase:

1. `app_users` : Usuarios del sistema (`id` , `name` , `email` , `password` , `role` , `active`).
2. `products` : Catálogo de mercancía (`id` , `sku` , `barcode` , `name` , `cost` , `price` , `stock` , `min_stock` , `unit`).
3. `categories` & `brands` : Clasificación del catálogo.
4. `customers` : Clientes y saldo en cartera (`id` , `name` , `document` , `phone` , `email` , `balance`).

5. `suppliers` : Proveedores del comercio.
 6. `cash_sessions` : Turnos de caja (`id` , `opening_amount` , `closing_amount` , `status` , `opened_at` , `closed_at`).
 7. `cash_movements` : Movimientos de efectivo (`id` , `type` , `amount` , `concept` , `reference` , `details`).
 8. `sales` & `sale_items` : Cabecera y detalle de facturas procesadas en POS.
 9. `purchases` & `purchase_items` : Órdenes de compra e ingreso de mercancía.
 10. `inventory_adjustments` : Auditoría de cambios de stock.
 11. `activity_logs` : Bitácora de auditoría global del sistema.
-

6. Módulos y Funcionalidades Destacadas



1. Punto de Venta (POS) & Resiliencia

- Cobro dinámico en Efectivo, Tarjeta, Transferencia y Crédito.
- Cálculo en tiempo real de subtotal, IVA y cambio.
- **Control de Caja (Opción A)**: Bloqueo de cobro si la caja está cerrada + ventana de apertura rápida en 1 clic.
- **Operatividad Ininterrumpida Offline**: Las ventas procesadas sin internet o con el servidor Supabase en mantenimiento se completan normalmente en pantalla y se encolan para su envío automático posterior.
- Generación e impresión de ticket térmico físico.



2. Catálogo e Inventario

- **Auto-generación de SKU corto**: Algoritmo que abrevia palabras a 3 letras y conserva unidades/números (ej. `ARR-ROA-1KG`).
- Desplegable de unidades de medida estándar (`pza` , `kg` , `lt` , `g` , `ml` , `caja` , `bot` , `lata` , `m` , `par` , `doc`).
- Creación ágil de nuevas categorías directamente dentro del formulario de productos.
- Alertas visuales de stock bajo.



3. Control de Caja y Cartera

- Registro automático de ingresos por ventas y egresos operativos.
- Arqueo de caja al cierre de turno.
- Gestión de abonos a cuentas por cobrar de clientes.



4. Auditoría y Permisos (Supervisor / Cajero)

- Matriz de permisos RBAC en `permissions.ts` por rol.

- Pestaña **Bitácora de Actividad** en Configuración para el Supervisor con buscador por fecha, usuario, acción y detalle.

7. Stack Tecnológico

Componente	Tecnología Seleccionada	Justificación Técnica
Core Framework	React 18 + Vite	Rendimiento óptimo, recarga rápida (HMR) y TypeScript nativo.
Estilos	Vanilla CSS + Tailwind CSS	Diseño responsivo, elegante (Dark/Light mode) y máxima personalización.
Animaciones	Framer Motion	Micro-interacciones fluidas y componentes interactivos premium.
Iconografía	Lucide React	Iconos vectoriales limpios y consistentes.
Persistencia & Resiliencia	Supabase (PostgreSQL) + Sync Engine LocalStorage	Persistencia remota en la nube con cola de sincronización resiliente ante caídas de red o servidor.

8. Guía para la Presentación Ejecutiva y Técnica

Al presentar **StoreFlow** ante clientes o evaluadores técnicos, destaca los siguientes 5 pilares:

- Alta Disponibilidad y Resiliencia Offline-First:** Demuestra cómo el sistema continúa vendiendo en el POS incluso desconectando el Wi-Fi o simulando una caída de Supabase, subiendo las ventas a la nube automáticamente al restablecer el servicio.
- Arquitectura Limpia (MVC + Service Layer):** Explica cómo la separación entre Vistas, Controladores y Servicios permite escalar o cambiar la base de datos sin romper la aplicación.
- Eficiencia Comercial (Demostrar el POS):** Enseña cómo realizar una venta en menos de 5 segundos, la auto-generación de SKU y el bloqueo inteligente cuando la caja está cerrada.
- Seguridad y Trazabilidad (Demostrar la Bitácora):** Muestra la pestaña de *Bitácora de Actividad* en Configuración para comprobar cómo cada acción queda auditada con nombre de usuario y timestamp.
- Diseño y Experiencia de Usuario (UI/UX):** Resalta la interfaz moderna, el modo oscuro/claro y la adaptabilidad a dispositivos.